# Dynamic arrays

## Growing an array in amortized constant time

Mohammed Said BELAID
said.belaid@univ-usto.dz
github.com/msbelaid

21/10/2026



# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# Back to the open question

At the end of the Hash Tables course:

- Our table has 10 slots and holds 10 students. Where does the 11th go?



# Back to the open question

At the end of the Hash Tables course:

- Our table has 10 slots and holds 10 students. Where does the 11th go?
- Answer: allocate a larger array and move the entries.



# Back to the open question

At the end of the Hash Tables course:

- Our table has 10 slots and holds 10 students. Where does the 11th go?
- Answer: allocate a larger array and move the entries.
- But how much larger? And what does it cost?



# Back to the open question

At the end of the Hash Tables course:

- Our table has 10 slots and holds 10 students. Where does the 11th go?
- Answer: allocate a larger array and move the entries.
- But how much larger? And what does it cost?



# Back to the open question

At the end of the Hash Tables course:

- Our table has 10 slots and holds 10 students. Where does the 11th go?
- Answer: allocate a larger array and move the entries.
- But how much larger? And what does it cost?

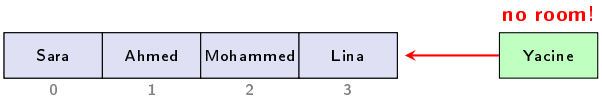
The same question arises for any array: today we answer it on the simplest case, a list of students.



# Arrays have a fixed size

```
String[] students = new String[4];

students[0] = "Sara";
students[1] = "Ahmed";
students[2] = "Mohammed";
students[3] = "Lina";
students[4] = "Yacine"; // ArrayIndexOutOfBoundsException
```




- The size is chosen at creation and cannot change.
- Students register one at a time: how many will come?



# Why not a linked list?

| Operation | Array | Linked list |
|---|---|---|
| Access by index | $O(1)$ | $O(n)$ |
| Append at the end | $O(1)$, if room is left | $O(1)$ (with a tail) |
| Memory | Contiguous cells | One node + pointer each |
| Cache behaviour | Excellent | Poor (scattered nodes) |



# Why not a linked list?

| Operation | Array | Linked list |
|---|---|---|
| Access by index | $O(1)$ | $O(n)$ |
| Append at the end | $O(1)$, if room is left | $O(1)$ (with a tail) |
| Memory | Contiguous cells | One node + pointer each |
| Cache behaviour | Excellent | Poor (scattered nodes) |

We want both:

- the index access and contiguous memory of an array,
- the unlimited growth of a linked list.



# The goal: a dynamic array

A dynamic array is a list backed by an array, which grows when needed.

| Operation | Meaning |
|---|---|
| `get(i)`, `set(i, v)` | read or replace the element at index $i$ |
| `add(v)` | append $v$ at the end |
| `add(i, v)` | insert $v$ at index $i$ |
| `remove(i)` | remove the element at index $i$ |
| `size()` | number of elements |



# The goal: a dynamic array

A dynamic array is a list backed by an array, which grows when needed.

| Operation | Meaning |
|---|---|
| `get(i)`, `set(i, v)` | read or replace the element at index $i$ |
| `add(v)` | append $v$ at the end |
| `add(i, v)` | insert $v$ at index $i$ |
| `remove(i)` | remove the element at index $i$ |
| `size()` | number of elements |

You already use it: `ArrayList` in Java, `list` in Python, `std::vector` in C++.



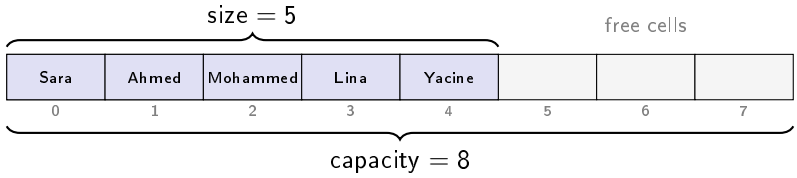
# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# Size vs. capacity

Idea: allocate more cells than needed.

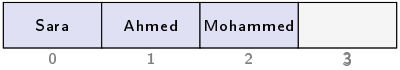



- Capacity: number of allocated cells (`data.length`).
- Size: number of elements actually stored.
- Always: $0 \leq \text{size} \leq \text{capacity}$.

The free cells are invisible to the user: `get(6)` is an error.

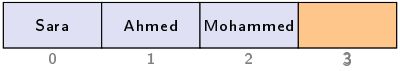


# Appending when there is room



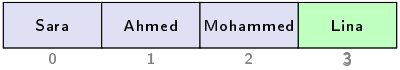



# Appending when there is room




`add("Lina")`: size $= 3 <$ capacity $= 4$: there is room at index `size`.

# Appending when there is room

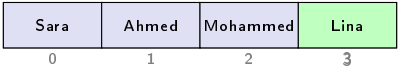



`add("Lina")`: size $= 3 <$ capacity $= 4$: there is room at index `size`.

Write `data[3] = "Lina"`, then `size = 4`.



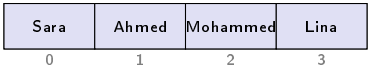
# Appending when there is room



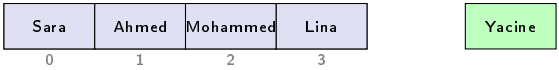

`add("Lina")`: size $= 3 <$ capacity $= 4$: there is room at index `size`.
Write `data[3] = "Lina"`, then `size = 4`.
Cost: one write, $O(1)$.



# Appending when the array is full

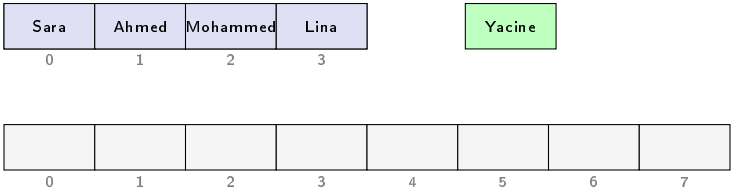


# Appending when the array is full




`add("Yacine")`: full

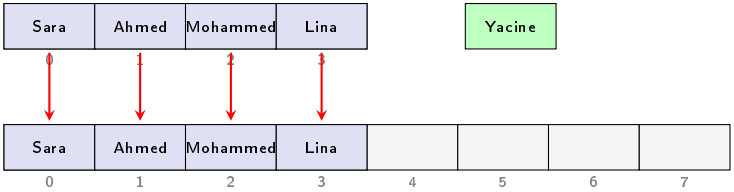
# Appending when the array is full




`add("Yacine")`: full

**1.** Allocate a new, bigger array

# Appending when the array is full



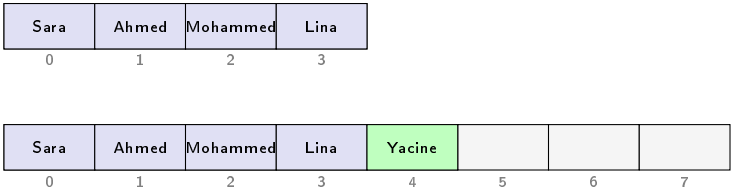

`add("Yacine")`: full

**1.** Allocate a new, bigger array

**2.** Copy the 4 elements



# Appending when the array is full

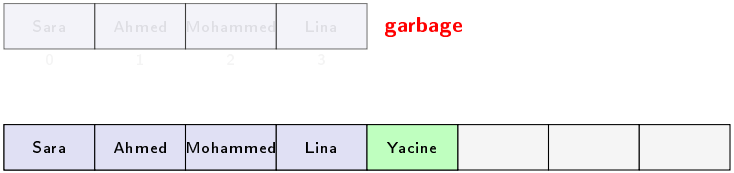



`add("Yacine")`: full

1. Allocate a new, bigger array
2. Copy the 4 elements
3. Write Yacine at index 4



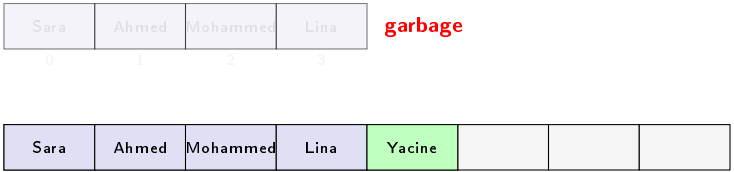
# Appending when the array is full




`add("Yacine")`: full

1. Allocate a new, bigger array
2. Copy the 4 elements
3. Write Yacine at index 4
4. Drop the old array (garbage collector)

# Appending when the array is full




`add("Yacine")`: full

1. Allocate a new, bigger array
2. Copy the 4 elements
3. Write Yacine at index 4
4. Drop the old array (garbage collector)

Cost: 4 copies + 1 write
$O(n)$



# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# How much bigger?

Each resize copies all the elements: $O(n)$.
Two natural choices for the new capacity:

- Strategy A: add a constant: capacity $+c$ (e.g. $+4$).
- Strategy B: multiply by a factor: capacity $\times 2$.



# How much bigger?

Each resize copies all the elements: $O(n)$.
Two natural choices for the new capacity:

- Strategy A: add a constant: capacity $+c$ (e.g. $+4$).
- Strategy B: multiply by a factor: capacity $\times 2$.

Let us count the copies for $n$ appends, starting from capacity 4.



# Strategy A: add a constant

Capacity: 4, 8, 12, 16, ... A resize happens every 4 appends.

$$\text{copies} = 4 + 8 + 12 + \cdots + (n-4) = 4\,(1 + 2 + \cdots + k) \approx \frac{n^2}{8} \quad \text{with } k \approx \frac{n}{4}$$



# Strategy A: add a constant

Capacity: 4, 8, 12, 16, ... A resize happens every 4 appends.

$$\text{copies} = 4 + 8 + 12 + \cdots + (n-4) = 4\,(1 + 2 + \cdots + k) \approx \frac{n^2}{8} \quad \text{with } k \approx \frac{n}{4}$$

- In general, with $+c$: about $\frac{n^2}{2c}$ copies: $O(n^2)$ for $n$ appends.
- A bigger $c$ only divides by a constant, and wastes more memory.
- Per append: $O(n)$ on average. As slow as inserting into a sorted array!



# Strategy B: double the capacity

Capacity: 4, 8, 16, 32, ... Resizes get rarer and rarer.

$$\text{copies} = 4 + 8 + 16 + \cdots + 2^j \quad \text{with } 2^j < n$$



# Strategy B: double the capacity

Capacity: 4, 8, 16, 32, ... Resizes get rarer and rarer.

$$\text{copies} = 4 + 8 + 16 + \cdots + 2^j \quad \text{with } 2^j < n$$

A geometric sum: each term is larger than all the previous ones together.

$$4 + 8 + \cdots + 2^j = 2^{j+1} - 4 < 2 \cdot 2^j < 2n$$



# Strategy B: double the capacity

Capacity: 4, 8, 16, 32, ... Resizes get rarer and rarer.

$$\text{copies} = 4 + 8 + 16 + \cdots + 2^j \quad \text{with } 2^j < n$$

A geometric sum: each term is larger than all the previous ones together.

$$4 + 8 + \cdots + 2^j = 2^{j+1} - 4 < 2 \cdot 2^j < 2n$$

- Fewer than $2n$ copies for $n$ appends: $O(n)$ in total.
- Per append: $O(1)$ on average!



# Comparing the two strategies

Total number of copies for $n$ appends, starting from capacity 4:

| $n$ | $+4$ | $\times 2$ |
|---|---|---|
| 16 | 24 | 12 |
| 1 024 | 130 560 | 1 020 |
| 1 000 000 | 124 999 500 000 | 1 048 572 |



# Comparing the two strategies

Total number of copies for $n$ appends, starting from capacity 4:

| $n$ | $+4$ | $\times 2$ |
|---|---|---|
| 16 | 24 | 12 |
| 1 024 | 130 560 | 1 020 |
| 1 000 000 | 124 999 500 000 | 1 048 572 |

At $\approx 10^9$ copies per second: 2 minutes vs. 1 millisecond.



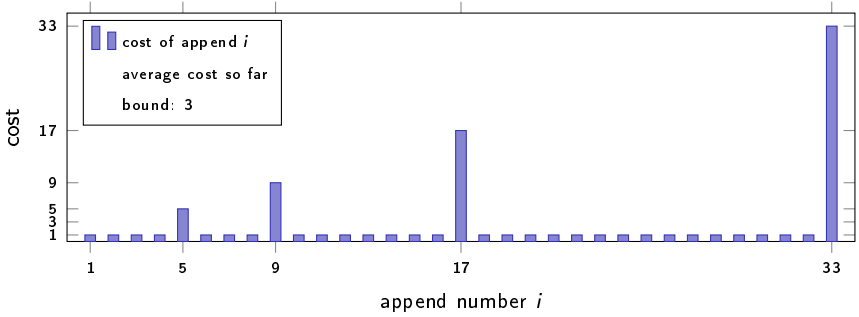
# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# The cost of each append

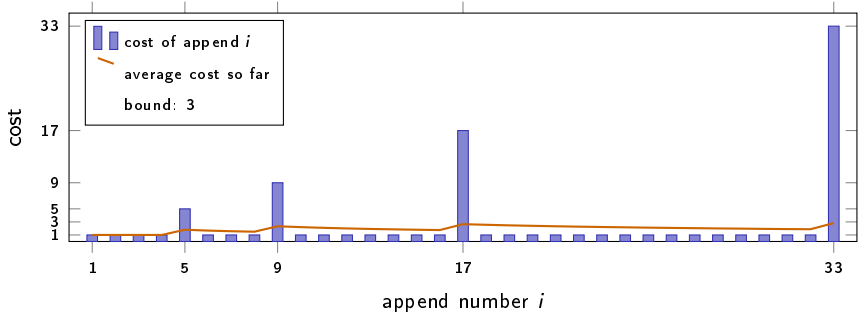
Cost of append #$i$ (writes + copies), doubling from capacity 4:




- Most appends cost 1; the expensive ones are rare and get further apart.

# The cost of each append

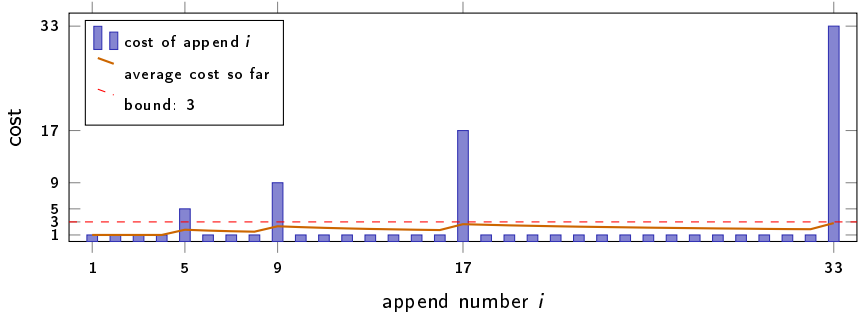
Cost of append $\#i$ (writes + copies), doubling from capacity 4:




- Most appends cost 1; the expensive ones are rare and get further apart.
- The average cost never exceeds 3.

# The cost of each append

Cost of append #$i$ (writes + copies), doubling from capacity 4:




- Most appends cost 1; the expensive ones are rare and get further apart.
- The average cost never exceeds 3.

# The aggregate method

Total cost of $n$ appends:

$$T(n) = \underbrace{n}_{\text{writes}} + \underbrace{4 + 8 + \cdots + 2^j}_{\text{copies} < 2n} < 3n \qquad \Rightarrow \qquad \frac{T(n)}{n} < 3$$

**Amortized cost**

The *amortized cost* of an operation is the total cost of a worst-case sequence of $n$ operations, divided by $n$.



# The aggregate method

Total cost of $n$ appends:

$$T(n) = \underbrace{n}_{\text{writes}} + \underbrace{4 + 8 + \cdots + 2^j}_{\text{copies} < 2n} < 3n \qquad \Rightarrow \qquad \frac{T(n)}{n} < 3$$

**Amortized cost**

The *amortized cost* of an operation is the total cost of a worst-case sequence of $n$ operations, divided by $n$.

Append in a doubling array: $O(1)$ amortized.



# Amortized, average, worst case

| Kind of bound | What is averaged? | Example |
|---|---|---|
| Worst case | nothing: the single worst operation | append: $O(n)$ |
| Average case | over the inputs (or hash values) | hash table get: $O(1)$ |
| Amortized | over a sequence of operations | append: $O(1)$ |



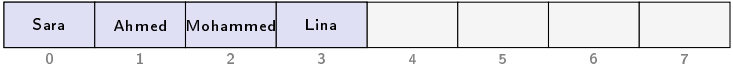
# Amortized, average, worst case

| Kind of bound | What is averaged? | Example |
|---|---|---|
| Worst case | nothing: the single worst operation | append: $O(n)$ |
| Average case | over the inputs (or hash values) | hash table get: $O(1)$ |
| Amortized | over a sequence of operations | append: $O(1)$ |

- The hash-table $O(1)$ assumes a good hash function: bad keys break it.
- The amortized $O(1)$ is a guarantee: it holds for any sequence. No probability.

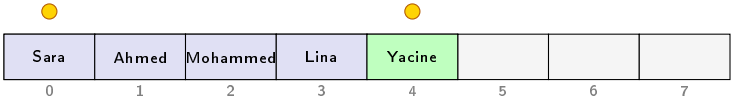


# The coins picture



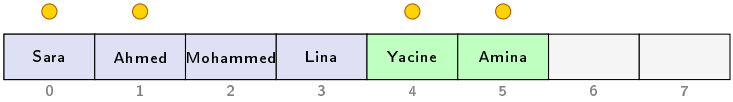

Just after a resize: capacity 8, half full, no coins saved.

# The coins picture




Just after a resize: capacity 8, half full, no coins saved.
Each append pays 3 coins: 1 for its write, 1 saved on itself, 1 saved on an old element.

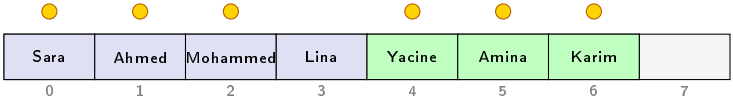
# The coins picture




Just after a resize: capacity 8, half full, no coins saved.
Each append pays 3 coins: 1 for its write, 1 saved on itself, 1 saved on an old element.

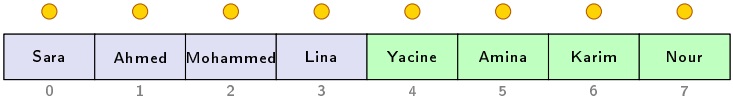


# The coins picture




Just after a resize: capacity 8, half full, no coins saved.
Each append pays 3 coins: 1 for its write, 1 saved on itself, 1 saved on an old element.

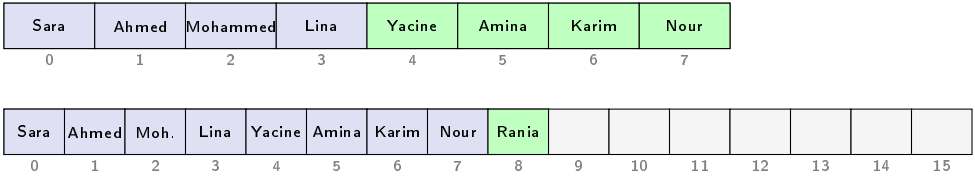
# The coins picture




Just after a resize: capacity 8, half full, no coins saved.
Each append pays 3 coins: 1 for its write, 1 saved on itself, 1 saved on an old element.
The array is full: every element holds exactly 1 coin.



# The coins picture




Just after a resize: capacity 8, half full, no coins saved.
Each append pays 3 coins: 1 for its write, 1 saved on itself, 1 saved on an old element.
The array is full: every element holds exactly 1 coin.
Append Rania: 8 copies, paid by the 8 coins. Back to 0 coins, half full.



# Why adding a constant fails

With capacity $+c$:

- After a resize, only $c$ appends happen before the next one.
- The next resize copies $n$ elements.
- To pay for it, each of the $c$ appends must save $\frac{n}{c}$ coins.



# Why adding a constant fails

With capacity $+c$:

- After a resize, only $c$ appends happen before the next one.
- The next resize copies $n$ elements.
- To pay for it, each of the $c$ appends must save $\frac{n}{c}$ coins.

The price grows with $n$: no constant number of coins is enough.



# Why adding a constant fails

With capacity $+c$:

- After a resize, only $c$ appends happen before the next one.
- The next resize copies $n$ elements.
- To pay for it, each of the $c$ appends must save $\frac{n}{c}$ coins.

The price grows with $n$: no constant number of coins is enough.
With doubling, $m/2$ appends separate two resizes, and the next resize copies $m$ elements: 2 coins per append always suffice.



# A single append can still be slow

- Amortized $O(1)$ does not mean that every append is $O(1)$: append $\#(2^k + 1)$ copies $2^k$ elements.
- A problem for latency-sensitive programs: games, real-time systems, user interfaces.



# A single append can still be slow

- Amortized $O(1)$ does not mean that every append is $O(1)$: append $\#(2^k + 1)$ copies $2^k$ elements.
- A problem for latency-sensitive programs: games, real-time systems, user interfaces.

If the final size is known, reserve the capacity upfront:

```
List<String> students = new ArrayList<>(100000); // no resize until 100000
```



# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# Implementation plan

1. Structure: an array and a size.
2. `get`, `set`: check the index against the size.
3. `add`: grow by doubling when full.
4. `add(i, v)`, `remove(i)`: shift the elements.
5. Test with our students.

Elements are `String` (student names).



# Structure

```
class DynamicArray {
    private String[] data; // (1)
    private int size;      // (2)

    DynamicArray() {
        data = new String[4]; // (3)
        size = 0;
    }

    int size()     { return size; }
    int capacity() { return data.length; } // (4)
}
```

1. The cells: occupied and free.



# Structure

```
class DynamicArray {
    private String[] data; // (1)
    private int size;      // (2)

    DynamicArray() {
        data = new String[4]; // (3)
        size = 0;
    }

    int size()     { return size; }
    int capacity() { return data.length; } // (4)
}
```

1. The cells: occupied and free.
2. The number of elements.



# Structure

```
class DynamicArray {
    private String[] data; // (1)
    private int size;      // (2)

    DynamicArray() {
        data = new String[4]; // (3)
        size = 0;
    }

    int size()     { return size; }
    int capacity() { return data.length; } // (4)
}
```

1. The cells: occupied and free.
2. The number of elements.
3. Initial capacity: 4, as in the drawings.



# Structure

```
class DynamicArray {
    private String[] data; // (1)
    private int size;      // (2)

    DynamicArray() {
        data = new String[4]; // (3)
        size = 0;
    }

    int size()     { return size; }
    int capacity() { return data.length; } // (4)
}
```

1. The cells: occupied and free.
2. The number of elements.
3. Initial capacity: 4, as in the drawings.
4. The capacity is the length of the array.



# get and set

```
String get(int index) {
    checkIndex(index);
    return data[index];
}

void set(int index, String value) {
    checkIndex(index);
    data[index] = value; // (1)
}

private void checkIndex(int index) {
    if (index < 0 || index >= size) { // (2)
        throw new IndexOutOfBoundsException(
            "Index " + index + ", size " + size);
    }
}
```

1. Direct array access: $O(1)$.



# get and set

```
String get(int index) {
    checkIndex(index);
    return data[index];
}

void set(int index, String value) {
    checkIndex(index);
    data[index] = value; // (1)
}

private void checkIndex(int index) {
    if (index < 0 || index >= size) { // (2)
        throw new IndexOutOfBoundsException(
            "Index " + index + ", size " + size);
    }
}
```

1. Direct array access: $O(1)$.
2. Compare with `size`, not `data.length`: the free cells are not elements.



# add and grow

```
void add(String value) {
    if (size == data.length) { // (1)
        grow();
    }
    data[size] = value; // (2)
    size++;
}

private void grow() {
    String[] bigger = new String[2 * data.length]; // (3)
    for (int i = 0; i < size; i++) {
        bigger[i] = data[i]; // (4)
    }
    data = bigger; // (5)
}
```

1. Full: make room first.



# add and grow

```
void add(String value) {
    if (size == data.length) { // (1)
        grow();
    }
    data[size] = value; // (2)
    size++;
}

private void grow() {
    String[] bigger = new String[2 * data.length]; // (3)
    for (int i = 0; i < size; i++) {
        bigger[i] = data[i]; // (4)
    }
    data = bigger; // (5)
}
```

1. Full: make room first.
2. The next free cell is at index `size`.



# add and grow

```
void add(String value) {
    if (size == data.length) { // (1)
        grow();
    }
    data[size] = value; // (2)
    size++;
}

private void grow() {
    String[] bigger = new String[2 * data.length]; // (3)
    for (int i = 0; i < size; i++) {
        bigger[i] = data[i]; // (4)
    }
    data = bigger; // (5)
}
```

1. Full: make room first.
2. The next free cell is at index `size`.
3. Double the capacity.



# add and grow

```
void add(String value) {
    if (size == data.length) { // (1)
        grow();
    }
    data[size] = value; // (2)
    size++;
}

private void grow() {
    String[] bigger = new String[2 * data.length]; // (3)
    for (int i = 0; i < size; i++) {
        bigger[i] = data[i]; // (4)
    }
    data = bigger; // (5)
}
```

1. Full: make room first.
2. The next free cell is at index `size`.
3. Double the capacity.
4. Copy the elements: $O(n)$.



# add and grow

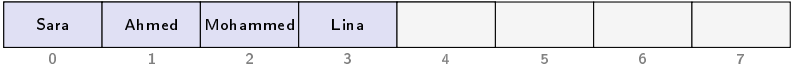
```
void add(String value) {
    if (size == data.length) { // (1)
        grow();
    }
    data[size] = value; // (2)
    size++;
}

private void grow() {
    String[] bigger = new String[2 * data.length]; // (3)
    for (int i = 0; i < size; i++) {
        bigger[i] = data[i]; // (4)
    }
    data = bigger; // (5)
}
```

1. Full: make room first.
2. The next free cell is at index `size`.
3. Double the capacity.
4. Copy the elements: $O(n)$.
5. The old array is now garbage.



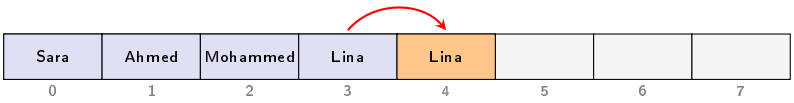
# Inserting in the middle: shifting




`add(1, "Yacine")`: index 1 is occupied by Ahmed.



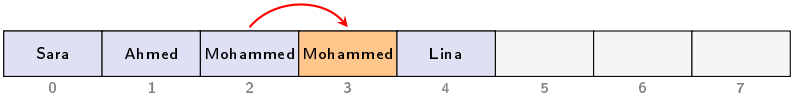
# Inserting in the middle: shifting




`add(1, "Yacine")`: index 1 is occupied by Ahmed.
Shift every element from index 1 one cell to the right, *starting from the end*.



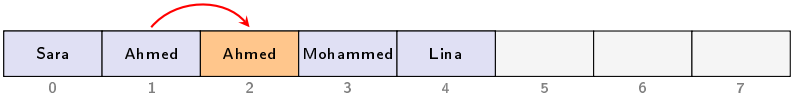
# Inserting in the middle: shifting




`add(1, "Yacine")`: index 1 is occupied by Ahmed.
Shift every element from index 1 one cell to the right, *starting from the end*.



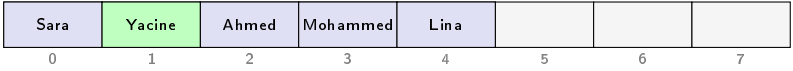
# Inserting in the middle: shifting




`add(1, "Yacine")`: index 1 is occupied by Ahmed.
Shift every element from index 1 one cell to the right, *starting from the end*.



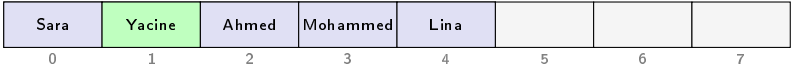
# Inserting in the middle: shifting




`add(1, "Yacine")`: index 1 is occupied by Ahmed.
Shift every element from index 1 one cell to the right, *starting from the end*.
Write Yacine at index 1; size $= 5$.



# Inserting in the middle: shifting




`add(1, "Yacine")`: index 1 is occupied by Ahmed.
Shift every element from index 1 one cell to the right, *starting from the end*.
Write Yacine at index 1; size $= 5$.
Cost: size $- i$ moves, $O(n)$. Inserting at index 0 moves everything.



# add(index, value) and remove(index)

```
void add(int index, String value) {
    if (index < 0 || index > size) { // (1)
        throw new IndexOutOfBoundsException();
    }
    if (size == data.length) {
        grow();
    }
    for (int i = size; i > index; i--) {
        data[i] = data[i - 1]; // (2)
    }
    data[index] = value;
    size++;
}

String remove(int index) {
    checkIndex(index);
    String removed = data[index];
    for (int i = index; i < size - 1; i++) {
        data[i] = data[i + 1]; // (3)
    }
    size--;
    data[size] = null; // (4)
    return removed;
}
```

1. `index = size` is allowed: it appends.



# add(index, value) and remove(index)

```
void add(int index, String value) {
    if (index < 0 || index > size) { // (1)
        throw new IndexOutOfBoundsException();
    }
    if (size == data.length) {
        grow();
    }
    for (int i = size; i > index; i--) {
        data[i] = data[i - 1]; // (2)
    }
    data[index] = value;
    size++;
}

String remove(int index) {
    checkIndex(index);
    String removed = data[index];
    for (int i = index; i < size - 1; i++) {
        data[i] = data[i + 1]; // (3)
    }
    size--;
    data[size] = null; // (4)
    return removed;
}
```

1. `index = size` is allowed: it appends.
2. Shift right, from the end.



# add(index, value) and remove(index)

```
void add(int index, String value) {
    if (index < 0 || index > size) { // (1)
        throw new IndexOutOfBoundsException();
    }
    if (size == data.length) {
        grow();
    }
    for (int i = size; i > index; i--) {
        data[i] = data[i - 1]; // (2)
    }
    data[index] = value;
    size++;
}

String remove(int index) {
    checkIndex(index);
    String removed = data[index];
    for (int i = index; i < size - 1; i++) {
        data[i] = data[i + 1]; // (3)
    }
    size--;
    data[size] = null; // (4)
    return removed;
}
```

1. `index = size` is allowed: it appends.
2. Shift right, from the end.
3. Shift left, from the removed index.



# add(index, value) and remove(index)

```
void add(int index, String value) {
    if (index < 0 || index > size) { // (1)
        throw new IndexOutOfBoundsException();
    }
    if (size == data.length) {
        grow();
    }
    for (int i = size; i > index; i--) {
        data[i] = data[i - 1]; // (2)
    }
    data[index] = value;
    size++;
}

String remove(int index) {
    checkIndex(index);
    String removed = data[index];
    for (int i = index; i < size - 1; i++) {
        data[i] = data[i + 1]; // (3)
    }
    size--;
    data[size] = null; // (4)
    return removed;
}
```

1. `index` = size= is allowed: it appends.
2. Shift right, from the end.
3. Shift left, from the removed index.
4. Clear the last cell, so the garbage collector can free the object.



# Test with the students

```
DynamicArray students = new DynamicArray();

students.add("Sara");
students.add("Ahmed");
students.add("Mohammed");
students.add("Lina");
System.out.println(students.size() + " / " + students.capacity());

students.add("Yacine"); // full: grow
System.out.println(students.size() + " / " + students.capacity());

students.add(1, "Amina");
System.out.println(students.get(1));
System.out.println(students.get(2));

students.remove(0);
System.out.println(students.get(0));
System.out.println(students.size() + " / " + students.capacity());
```

Expected output:

```
4 / 4
5 / 8
Amina
Ahmed
Amina
5 / 8
```



# Cost of the operations

| Operation | Cost |
|---|---|
| `get(i)`, `set(i, v)` | $O(1)$ |
| `add(v)` | $O(1)$ amortized, $O(n)$ worst case |
| `add(i, v)` | $O(n)$: shifting |
| `remove(i)` | $O(n)$: shifting |
| remove the last | $O(1)$ |



# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# Wasted memory

```
for (int i = 0; i < 1000; i++) students.add("Student " + i);
for (int i = 0; i < 990; i++)  students.remove(students.size() - 1);
```

- Size: 10, capacity: 1024. 99% of the cells are wasted.



# Wasted memory

```
for (int i = 0; i < 1000; i++) students.add("Student " + i);
for (int i = 0; i < 990; i++)  students.remove(students.size() - 1);
```

- Size: 10, capacity: 1024. 99% of the cells are wasted.

Idea: halve the capacity when the array becomes sparse.



# Wasted memory

```
for (int i = 0; i < 1000; i++) students.add("Student " + i);
for (int i = 0; i < 990; i++)  students.remove(students.size() - 1);
```

- Size: 10, capacity: 1024. 99% of the cells are wasted.

Idea: halve the capacity when the array becomes sparse.
When exactly? The natural answer, "when it is half full", is a trap.



# First idea: shrink when half full

Grow $\times 2$ when full, shrink $\div 2$ when size $=$ capacity $/2$. Start: full, size $=$ capacity $= 4$.

| Operation | Size | Capacity | Copies |
|---|---|---|---|



# First idea: shrink when half full

Grow $\times 2$ when full, shrink $\div 2$ when size $=$ capacity $/2$. Start: full, size $=$ capacity $= 4$.

| Operation | Size | Capacity | Copies |
|---|---|---|---|
| add: full, grow | 5 | 8 | 4 |



# First idea: shrink when half full

Grow $\times 2$ when full, shrink $\div 2$ when size $=$ capacity $/2$. Start: full, size $=$ capacity $= 4$.

| Operation | Size | Capacity | Copies |
|---|---|---|---|
| add: full, grow | 5 | 8 | 4 |
| remove: half, shrink | 4 | 4 | 4 |



# First idea: shrink when half full

Grow $\times 2$ when full, shrink $\div 2$ when size $=$ capacity $/2$. Start: full, size $=$ capacity $= 4$.

| Operation | Size | Capacity | Copies |
|---|---|---|---|
| add: full, grow | 5 | 8 | 4 |
| remove: half, shrink | 4 | 4 | 4 |
| add: full, grow | 5 | 8 | 4 |



# First idea: shrink when half full

Grow $\times 2$ when full, shrink $\div 2$ when size $=$ capacity $/2$. Start: full, size $=$ capacity $= 4$.

| Operation | Size | Capacity | Copies |
|---|---|---|---|
| add: full, grow | 5 | 8 | 4 |
| remove: half, shrink | 4 | 4 | 4 |
| add: full, grow | 5 | 8 | 4 |
| remove: half, shrink | 4 | 4 | 4 |



# First idea: shrink when half full

Grow $\times 2$ when full, shrink $\div 2$ when size $=$ capacity $/2$. Start: full, size $=$ capacity $= 4$.

| Operation | Size | Capacity | Copies |
|---|---|---|---|
| add: full, grow | 5 | 8 | 4 |
| remove: half, shrink | 4 | 4 | 4 |
| add: full, grow | 5 | 8 | 4 |
| remove: half, shrink | 4 | 4 | 4 |
| ... | | | |



# First idea: shrink when half full

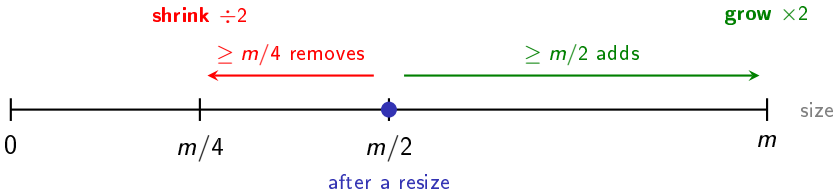
Grow $\times 2$ when full, shrink $\div 2$ when size $=$ capacity $/2$. Start: full, size $=$ capacity $= 4$.

| Operation | Size | Capacity | Copies |
|---|---|---|---|
| add: full, grow | 5 | 8 | 4 |
| remove: half, shrink | 4 | 4 | 4 |
| add: full, grow | 5 | 8 | 4 |
| remove: half, shrink | 4 | 4 | 4 |
| ... | | | |

Every operation copies $n$ elements: $O(n)$ each. This is thrashing.



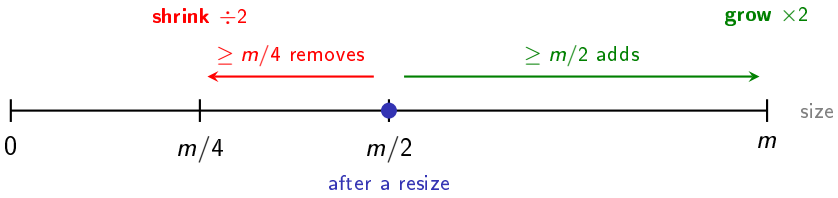
# Solution: shrink when a quarter full




- Grow $\times 2$ when full; shrink $\div 2$ when size $=$ capacity $/4$.
- After any resize, the array is exactly half full.
- At least $m/4$ operations happen before the next resize: they save enough coins.



# Solution: shrink when a quarter full




- Grow $\times 2$ when full; shrink $\div 2$ when size $=$ capacity $/4$.
- After any resize, the array is exactly half full.
- At least $m/4$ operations happen before the next resize: they save enough coins.

$\Rightarrow O(1)$ amortized for any sequence of adds and removes.



# Exercise: shrinking

Modify `remove(index)` so that the array shrinks:

- halve the capacity when size = capacity /4,
- never go below a minimum capacity of 4.

Test it: add 1000 students, remove 990 of them, then print the size and the capacity.



# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# The table is too full

Hash tables have the same problem: the load factor $\alpha = n/m$ grows.

- Java's `HashMap` doubles its number of buckets when $\alpha > 0.75$.
- Same idea as the dynamic array: grow by a factor.



# The table is too full

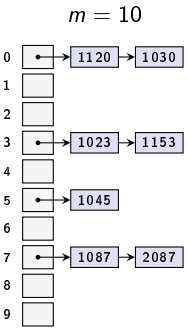
Hash tables have the same problem: the load factor $\alpha = n/m$ grows.

- Java's `HashMap` doubles its number of buckets when $\alpha > 0.75$.
- Same idea as the dynamic array: grow by a factor.

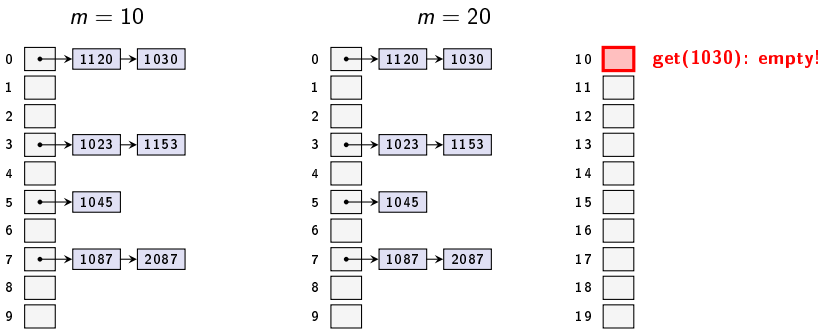
Can we simply copy the buckets into a bigger array?



# Copying is not enough



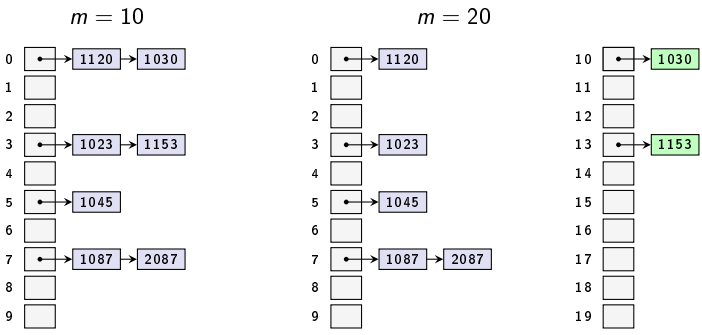
# Copying is not enough




Naive copy: `get(1030)` computes $1030 \bmod 20 = 10$; bucket 10 is empty: `null`!



# Copying is not enough




Naive copy: `get(1030)` computes $1030 \bmod 20 = 10$; bucket 10 is empty: `null`!

*Rehashing*: re-insert every entry with the new $h(k) = k \bmod 20$.



# The cost of rehashing

- Rehashing re-inserts all $n$ entries: $O(n)$, like copying an array.
- It happens when $\alpha$ crosses the threshold, and $m$ doubles each time.
- Same aggregate argument: put is $O(1)$ amortized (and on average, for a good hash function).



# The cost of rehashing

- Rehashing re-inserts all $n$ entries: $O(n)$, like copying an array.
- It happens when $\alpha$ crosses the threshold, and $m$ doubles each time.
- Same aggregate argument: put is $O(1)$ amortized (and on average, for a good hash function).

| Language | Initial buckets | Resize when | Growth |
|---|---|---|---|
| Java `HashMap` | 16 | $\alpha > 0.75$ | $\times 2$ |
| Python `dict` | 8 | $\alpha > 2/3$ | $\times 2$ or more |



# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# Java ArrayList

```
List<String> students = new ArrayList<>();
students.add("Sara");           // append: O(1) amortized
students.add(0, "Ahmed");       // insert at the front: O(n)
students.get(1);                // Sara
students.remove(0);             // O(n)

ArrayList<String> big = new ArrayList<>(100000); // reserve the capacity
big.ensureCapacity(200000);     // grow once, now
big.trimToSize();               // capacity = size
```

- Grows by $\times 1.5$: `newCapacity = old + (old >> 1)`.
- Never shrinks by itself: call `trimToSize()`.



# Python list and C++ vector

## Python

```python
students = []                       # list
students.append("Sara")             # O(1) amortized
students.insert(0, "Ahmed")         # O(n)
students.pop()                      # last: O(1)
students.pop(0)                     # first: O(n)

import sys
sys.getsizeof(students)   # grows in steps
```

## C++

```cpp
std::vector<std::string> students;
students.reserve(100);              // capacity >= 100
students.push_back("Sara");         // O(1) amortized
students.size();                    // 1
students.capacity();                // 100
students.pop_back();                // O(1)
students.shrink_to_fit();           // request
```

- Python grows by $\approx \times 1.125$; C++: $\times 2$ (GCC), $\times 1.5$ (MSVC).



# Choosing the growth factor

Any factor $r > 1$ gives $O(1)$ amortized. The constant changes:

| Factor $r$ | Copies per append (amortized) | Wasted cells after a resize |
|---|---|---|
| 2 | $\leq 1$ | up to 50% |
| 1.5 | $\leq 2$ | up to 33% |
| 1.125 | $\leq 8$ | up to 11% |



# Choosing the growth factor

Any factor $r > 1$ gives $O(1)$ amortized. The constant changes:

| Factor $r$ | Copies per append (amortized) | Wasted cells after a resize |
|---|---|---|
| 2 | $\leq 1$ | up to 50% |
| 1.5 | $\leq 2$ | up to 33% |
| 1.125 | $\leq 8$ | up to 11% |

A trade-off between time (fewer copies) and memory (fewer free cells).



# Dynamic array vs. linked list

| Operation | Dynamic array | Linked list |
|---|---|---|
| Access by index | $O(1)$ | $O(n)$ |
| Append at the end | $O(1)$ amortized | $O(1)$ |
| Insert at the front | $O(n)$ | $O(1)$ |
| Insert after a known node | $O(n)$ | $O(1)$ |
| Extra memory | Free cells | One pointer per element |
| Cache behaviour | Excellent | Poor |



# Outline

1 The problem: a fixed-size array
2 Size and capacity
3 Growth strategies
4 Amortized analysis
5 Implementing a dynamic array
6 Shrinking
7 Back to hash tables: rehashing
8 Dynamic arrays in practice
9 Summary



# Key takeaways

- A dynamic array separates size and capacity.
- Growing copies everything, $O(n)$: grow by a factor, never by a constant.
- Doubling makes append $O(1)$ amortized: a guarantee over any sequence.
- Shrink at 1/4, not 1/2, to avoid thrashing.
- Hash tables grow the same way: rehashing re-inserts every entry.
- In practice: reserve the capacity when the size is known.



# Open question: searching by prefix

A hash table holds all the words of a dictionary.

- `get("algorithm")`: $O(1)$ on average. Perfect.



# Open question: searching by prefix

A hash table holds all the words of a dictionary.

- `get("algorithm")`: $O(1)$ on average. Perfect.
- Autocomplete: find all the words starting with `"algo"`.



# Open question: searching by prefix

A hash table holds all the words of a dictionary.

- `get("algorithm")`: $O(1)$ on average. Perfect.
- Autocomplete: find all the words starting with `"algo"`.
- Where are these words in the table? Is a hash table still a good choice?

